

Министерство цифрового развития, связи и массовых
коммуникаций Российской Федерации
Федеральное государственное бюджетное общеобразовательное
учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»
(СибГУТИ)

Расчетно-графическая работа
по дисциплине: «Базы данных»

Выполнили:
Демин С. А. и Петров Р. Ю.
Группа: ИКС-433
Проверил: Брагин К. И.

Новосибирск, 2026

Оглавление

1. Цель работы	4
2. Выполнение работы	5
2.0 Подготовка и описание предметной области	5
Создали GitHub: https://github.com/seg0ga/db_rgr	5
2.1 Создание схемы БД (schema.sql)	6
2.1.1 Создание таблиц	6
2.1.2 Добавление внешних ключей	8
2.1.3 Создание индексов	8
2.2 Наполнение тестовыми данными (data.sql)	9
2.2.1 Статистика наполнения	9
2.2.2 Примеры добавленных данных	10
2.3 Визуализация структуры БД (diagram.png).....	13
2.4 Хранимые функции (functions.sql).....	14
Функция 1: Подсчёт активных инцидентов на сервере	14
Функция 2: Расчёт энергопотребления стойки	15
Функция 3: Суммарное время простоя за период.....	15
Функция 4: Проверка возможности добавления сервера в стойку.....	16
2.5 Триггеры (triggers.sql)	17
Триггер 1: Аудит изменения статуса сервера	17
Триггер 2: Автообновление метки времени updated_at	18
2.6 Демонстрационный сценарий (demo.sql)	18
Блок 1: Проверка наполненности базы	18
Блок 2: Вывод структуры, ограничений и индексов	19
Блок 3: Связи серверов со стойками, статусами и сервисами	20
Блок 4: Инциденты с приоритетами.....	21
Блок 5: Аналитические запросы	21
Блок 6: Вызов хранимых функций	23
Блок 7: Демонстрация работы триггеров.....	23
Блок 8: Проверка ON DELETE CASCADE	24
2.7 Аналитические запросы (queries.sql).....	25
2.7.1 Простые SELECT-запросы (6 шт.)	25
2.7.2 Запросы с фильтрацией WHERE (4 шт.)	27
2.7.3 Запросы с JOIN (4 шт.)	28
2.7.4 Запросы с агрегацией GROUP BY (6 шт.)	30
2.7.5 Запросы с HAVING (2 шт.)	32

2.7.6 Запросы с CTE (2 шт.).....	32
2.7.7 Запросы с ORDER BY + LIMIT (2 шт.)	33
2.8 Запуск через Docker Compose.....	34
2.8.1 Файл конфигурации	34
2.8.2 Порядок запуска	35
2.8.3 Автоматическая инициализация.....	35
2.9 Результаты выполнения	35
3. Выводы	37

1. Цель работы

1.1 Спроектировать авторскую реляционную базу данных по тематике, связанной с IT-инфраструктурой дата-центра (стойки, серверы, сервисы, инциденты).

1.2 Реализовать спроектированную схему в СУБД PostgreSQL с использованием структурированного SQL-скрипта.

1.3 Визуализировать логическую структуру БД в виде диаграммы (ER-модели).

1.4 Разработать комплексный набор SQL-запросов, демонстрирующих функциональные возможности и аналитический потенциал созданной системы.

1.5 Освоить продвинутые механизмы PostgreSQL: хранимые функции, триггеры аудита, контейнеризацию через Docker Compose.

2. Выполнение работы

2.0 Подготовка и описание предметной области

Создали GitHub: https://github.com/seg0ga/db_rgr

Спроектирована база данных для централизованного учёта и мониторинга IT-инфраструктуры дата-центра. Предметная область включает следующие ключевые сущности и бизнес-процессы:

- **Стойки (racks)** - физическое размещение оборудования в дата-центрах DC-1, DC-2 и DR-сайте. Каждая стойка имеет уникальное имя, локацию и максимальную энергомощность.
- **Статусы серверов (server_statuses)** - справочник состояний: ACTIVE (активный), MAINTENANCE (в обслуживании), OFFLINE (отключённый), PROVISIONING (подготовка к работе), DECOMMISSIONED (выведен из эксплуатации).
- **Серверы (servers)** - основная сущность, содержащая hostname, модель, количество CPU/ОЗУ, IP-адрес, серийный номер, ОС, дату покупки и привязку к стойке и статусу.
- **Сервисы (services)** - программные сервисы, работающие на серверах (billing-api, postgres-primary, redis-cache и др.), с указанием порта и статуса.
- **Приоритеты инцидентов (incident_priorities)** - справочник уровней критичности (LOW, MEDIUM, HIGH, CRITICAL) с нормативным временем реакции.
- **Инциденты (incidents)** - журнал сбоев и проблем, привязанный к серверу и опционально к сервису, с временем обнаружения и устранения.
- **Журнал обслуживания (maintenance_log)** - регистрация плановых и внеплановых работ: обновления ОС, прошивки, замена памяти, диагностика и т.д., с фиксацией времени простоя.
- **Аудит статусов серверов (servers_audit)** - автоматическая запись истории изменений статуса с указанием времени и инициатора.

Бизнес-правила, заложенные в схему:

- Сервер не может существовать без стойки и статуса.
- При удалении сервера все его сервисы удаляются автоматически (ON DELETE CASCADE).

- Инцидент может быть привязан к серверу обязательно, а к сервису - опционально.
- Время разрешения инцидента не может быть раньше времени обнаружения.
- Все числовые параметры (мощность, CPU, RAM, порт, время простоя) имеют проверки на допустимые значения.

2.1 Создание схемы БД (schema.sql)

Разработан единый скрипт schema.sql, который создаёт все таблицы, ограничения и индексы. Ниже приведены основные фрагменты с пояснениями.

2.1.1 Создание таблиц

Таблица стоек:

```
CREATE TABLE racks (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE,
  location VARCHAR(100) NOT NULL,
  power_capacity INT CHECK (power_capacity > 0)
);
```

Таблица статусов серверов:

```
CREATE TABLE server_statuses (
  id SERIAL PRIMARY KEY,
  status_name VARCHAR(20) UNIQUE NOT NULL
);
```

Таблица серверов:

```
CREATE TABLE servers (
  id SERIAL PRIMARY KEY,
  hostname VARCHAR(100) UNIQUE NOT NULL,
  rack_id INT NOT NULL,
  model VARCHAR(50) NOT NULL,
  cpu_cores INT CHECK (cpu_cores > 0),
  ram_gb INT CHECK (ram_gb > 0),
  status_id INT NOT NULL,
  purchase_date DATE,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  ip_address INET UNIQUE,
  serial_number VARCHAR(100),
  os_name VARCHAR(100)
);
```

Таблица сервисов:

```
CREATE TABLE services (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  server_id INT NOT NULL,
  port INT CHECK (port BETWEEN 1 AND 65535),
  status VARCHAR(20) DEFAULT 'RUNNING'
);
```

Таблица приоритетов инцидентов:

```
CREATE TABLE incident_priorities (  
  id SERIAL PRIMARY KEY,  
  level VARCHAR(10) UNIQUE NOT NULL,  
  response_minutes INT  
);
```

Таблица инцидентов:

```
CREATE TABLE incidents (  
  id SERIAL PRIMARY KEY,  
  server_id INT NOT NULL,  
  service_id INT,  
  priority_id INT NOT NULL,  
  title VARCHAR(200) NOT NULL,  
  detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  resolved_at TIMESTAMP,  
  CHECK (resolved_at IS NULL OR resolved_at >= detected_at)  
);
```

Таблица журнала обслуживания:

```
CREATE TABLE maintenance_log (  
  id SERIAL PRIMARY KEY,  
  server_id INT NOT NULL,  
  task_type VARCHAR(50) NOT NULL,  
  performed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  description TEXT,  
  downtime_minutes INT CHECK (downtime_minutes >= 0)  
);
```

Таблица аудита статусов:

```
CREATE TABLE servers_audit (  
  id SERIAL PRIMARY KEY,  
  server_id INT,  
  old_status_id INT,  
  new_status_id INT,  
  changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  changed_by VARCHAR(50) DEFAULT CURRENT_USER  
);
```

```
dc_equipment_rgr=# \dt
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | incident_priorities | table | rgr_user
public | incidents | table | rgr_user
public | maintenance_log | table | rgr_user
public | racks | table | rgr_user
public | server_statuses | table | rgr_user
public | servers | table | rgr_user
public | servers_audit | table | rgr_user
public | services | table | rgr_user
(8 rows)
```

2.1.2 Добавление внешних ключей

```
ALTER TABLE servers ADD CONSTRAINT fk_servers_rack FOREIGN KEY (rack_id)
REFERENCES racks(id);
```

```
ALTER TABLE servers ADD CONSTRAINT fk_servers_status FOREIGN KEY
(status_id) REFERENCES server_statuses(id);
```

```
ALTER TABLE servers_audit ADD CONSTRAINT fk_audit_server FOREIGN KEY
(server_id) REFERENCES servers(id);
```

```
ALTER TABLE servers_audit ADD CONSTRAINT fk_audit_old_status FOREIGN KEY
(old_status_id) REFERENCES server_statuses(id);
```

```
ALTER TABLE servers_audit ADD CONSTRAINT fk_audit_new_status FOREIGN KEY
(new_status_id) REFERENCES server_statuses(id);
```

```
ALTER TABLE services ADD CONSTRAINT fk_services_server FOREIGN KEY
(server_id) REFERENCES servers(id) ON DELETE CASCADE;
```

```
ALTER TABLE incidents ADD CONSTRAINT fk_incidents_server FOREIGN KEY
(server_id) REFERENCES servers(id);
```

```
ALTER TABLE incidents ADD CONSTRAINT fk_incidents_service FOREIGN KEY
(service_id) REFERENCES services(id);
```

```
ALTER TABLE incidents ADD CONSTRAINT fk_incidents_priority FOREIGN KEY
(priority_id) REFERENCES incident_priorities(id);
```

```
ALTER TABLE maintenance_log ADD CONSTRAINT fk_maintenance_server
FOREIGN KEY (server_id) REFERENCES servers(id);
```

2.1.3 Создание индексов

```
CREATE INDEX idx_servers_rack_id ON servers(rack_id);
```

```
CREATE INDEX idx_servers_status_id ON servers(status_id);
```

```
CREATE INDEX idx_servers_hostname ON servers(hostname);
```

```
CREATE INDEX idx_servers_ip_address ON servers(ip_address);
```

```
CREATE INDEX idx_services_server_id ON services(server_id);
```

```
CREATE INDEX idx_services_status ON services(status);
```

```
CREATE INDEX idx_incidents_detected_at ON incidents(detected_at);
```



```
CREATE INDEX idx_incidents_priority_id ON incidents(priority_id);
CREATE INDEX idx_incidents_server_id ON incidents(server_id);
CREATE INDEX idx_maintenance_log_performed_at ON
maintenance_log(performed_at);
CREATE INDEX idx_maintenance_log_server_id ON maintenance_log(server_id);
CREATE INDEX idx_servers_audit_changed_at ON servers_audit(changed_at);
CREATE INDEX idx_servers_audit_server_id ON servers_audit(server_id);
```

```
dc_equipments_rgr=# \di
```

List of relations				
Schema	Name	Type	Owner	Table
public	idx_incidents_detected_at	index	rgr_user	incidents
public	idx_incidents_priority_id	index	rgr_user	incidents
public	idx_incidents_server_id	index	rgr_user	incidents
public	idx_maintenance_log_performed_at	index	rgr_user	maintenance_log
public	idx_maintenance_log_server_id	index	rgr_user	maintenance_log
public	idx_servers_audit_changed_at	index	rgr_user	servers_audit
public	idx_servers_audit_server_id	index	rgr_user	servers_audit
public	idx_servers_hostname	index	rgr_user	servers
public	idx_servers_ip_address	index	rgr_user	servers
public	idx_servers_rack_id	index	rgr_user	servers
public	idx_servers_status_id	index	rgr_user	servers
public	idx_services_server_id	index	rgr_user	services
public	idx_services_status	index	rgr_user	services
public	incident_priorities_level_key	index	rgr_user	incident_priorities
public	incident_priorities_pkey	index	rgr_user	incident_priorities
public	incidents_pkey	index	rgr_user	incidents
public	maintenance_log_pkey	index	rgr_user	maintenance_log
public	racks_name_key	index	rgr_user	racks
public	racks_pkey	index	rgr_user	racks
public	server_statuses_pkey	index	rgr_user	server_statuses
public	server_statuses_status_name_key	index	rgr_user	server_statuses
public	servers_audit_pkey	index	rgr_user	servers_audit
public	servers_hostname_key	index	rgr_user	servers
public	servers_ip_address_key	index	rgr_user	servers
public	servers_pkey	index	rgr_user	servers
public	services_pkey	index	rgr_user	services

```
(26 rows)
```

2.2 Наполнение тестовыми данными (data.sql)

Скрипт data.sql наполняет базу репрезентативными данными для демонстрации всех функциональных возможностей.

2.2.1 Статистика наполнения

Таблица	Количество записей
racks	8
server_statuses	5
incident_priorities	4
servers	20
services	24
incidents	18
maintenance_log	18
servers_audit	(заполняется триггерами)
Итого	97+

2.2.2 Примеры добавленных данных

Стойки:

```
INSERT INTO racks (id, name, location, power_capacity) VALUES
```

```
(1, 'RACK-A01', 'DC-1 / Hall A / Row 1', 12000),  
(2, 'RACK-A02', 'DC-1 / Hall A / Row 1', 12000),  
(4, 'RACK-B01', 'DC-1 / Hall B / Row 1', 18000),  
(6, 'RACK-C01', 'DC-2 / Hall C / Row 1', 20000),  
(8, 'RACK-D01', 'DR-Site / Hall D / Row 1', 10000);
```

```
dc_equipment_rgr=# SELECT * FROM racks;  
 id | name | location | power_capacity  
----+-----+-----+-----  
  1 | RACK-A01 | DC-1 / Hall A / Row 1 | 12000  
  2 | RACK-A02 | DC-1 / Hall A / Row 1 | 12000  
  3 | RACK-A03 | DC-1 / Hall A / Row 2 | 16000  
  4 | RACK-B01 | DC-1 / Hall B / Row 1 | 18000  
  5 | RACK-B02 | DC-1 / Hall B / Row 1 | 18000  
  6 | RACK-C01 | DC-2 / Hall C / Row 1 | 20000  
  7 | RACK-C02 | DC-2 / Hall C / Row 2 | 20000  
  8 | RACK-D01 | DR-Site / Hall D / Row 1 | 10000  
(8 rows)
```

Статусы серверов:

```
INSERT INTO server_statuses (id, status_name) VALUES
```

```
(1, 'ACTIVE'),  
(2, 'MAINTENANCE'),  
(3, 'OFFLINE'),  
(4, 'PROVISIONING'),  
(5, 'DECOMMISSIONED');
```

```
dc_equipment_rgr=# SELECT * FROM server_statuses;  
 id | status_name  
----+-----  
  1 | ACTIVE  
  2 | MAINTENANCE  
  3 | OFFLINE  
  4 | PROVISIONING  
  5 | DECOMMISSIONED  
(5 rows)
```

Приоритеты инцидентов:

```
INSERT INTO incident_priorities (id, level, response_minutes) VALUES
```

```
(1, 'LOW', 1440),  
(2, 'MEDIUM', 240),  
(3, 'HIGH', 60),  
(4, 'CRITICAL', 15);
```

```
dc_equipment_rgr=# SELECT * FROM incident_priorities;
id | level | response_minutes
---+-----+-----
1 | LOW | 1440
2 | MEDIUM | 240
3 | HIGH | 60
4 | CRITICAL | 15
(4 rows)
```

Серверы (примеры):

INSERT INTO servers (id, hostname, rack_id, model, cpu_cores, ram_gb, status_id, purchase_date, ip_address, os_name) VALUES
(1, 'srv-app-01', 1, 'Dell R750', 32, 128, 1, '2024-03-14', '10.10.1.11', 'Ubuntu Server 24.04'),
(5, 'srv-db-01', 2, 'Dell R760', 64, 512, 1, '2025-01-19', '10.10.2.31', 'Rocky Linux 9'),
(12, 'srv-ai-01', 4, 'Dell XE9680', 128, 1024, 1, '2026-02-04', '10.10.4.81', 'Ubuntu Server 24.04'),
(20, 'srv-legacy-01', 8, 'IBM x3650', 16, 64, 5, '2020-04-17', '10.20.1.12', 'CentOS 7');

dc_equipment_rgr=# SELECT * FROM servers;											
id	hostname	rack_id	model	cpu_cores	ram_gb	status_id	purchase_date	updated_at	ip_address	serial_number	os_name
1	srv-app-01	1	Dell R750	32	128	1	2024-03-14	2026-05-10 09:30:00	10.10.1.11	DL750-A001	Ubuntu Server 24.04
2	srv-app-02	1	Dell R750	32	128	1	2024-03-14	2026-05-11 10:00:00	10.10.1.12	DL750-A002	Ubuntu Server 24.04
3	srv-web-01	1	HPE DL360	24	64	1	2023-11-02	2026-04-29 08:15:00	10.10.1.21	HP360-W001	Debian 12
4	srv-web-02	1	HPE DL360	24	64	2	2023-11-02	2026-02-01 12:00:00	10.10.1.22	HP360-W002	Debian 12
5	srv-db-01	2	Dell R760	64	512	1	2025-01-19	2026-05-14 03:20:00	10.10.2.31	DL760-D001	Rocky Linux 9
6	srv-db-02	2	Dell R760	64	512	1	2025-01-19	2026-05-14 03:45:00	10.10.2.32	DL760-D002	Rocky Linux 9
7	srv-cache-01	2	Lenovo SR650	32	256	1	2024-08-08	2026-05-01 17:10:00	10.10.2.41	LN650-C001	Ubuntu Server 22.04
8	srv-cache-02	2	Lenovo SR650	32	256	3	2024-08-08	2026-01-05 07:00:00	10.10.2.42	LN650-C002	Ubuntu Server 22.04
9	srv-mon-01	3	Supermicro SYS-120	16	64	1	2023-05-30	2026-05-13 16:40:00	10.10.3.51	SM120-M001	Debian 12
10	srv-log-01	3	Supermicro SYS-220	32	256	1	2024-02-12	2026-04-21 06:20:00	10.10.3.61	SM220-L001	Rocky Linux 9
11	srv-bak-01	3	HPE DL380	48	384	1	2024-06-18	2026-03-25 23:30:00	10.10.3.71	HP380-B001	Ubuntu Server 24.04
12	srv-ai-01	4	Dell XE9680	128	1024	1	2026-02-04	2026-05-12 14:05:00	10.10.4.81	DLX9680-A101	Ubuntu Server 24.04
13	srv-ai-02	4	Dell XE9680	128	1024	4	2026-02-04	2026-05-15 11:15:00	10.10.4.82	DLX9680-A102	Ubuntu Server 24.04
14	srv-vpn-01	5	HPE DL360	16	64	1	2022-09-10	2026-01-20 18:00:00	10.10.5.01	HP360-V001	Debian 11
15	srv-edge-01	5	Lenovo SE350	12	32	1	2026-01-12	2026-05-05 09:20:00	10.10.5.101	LN350-E001	Ubuntu Server 24.04
16	srv-edge-02	5	Lenovo SE350	12	32	1	2026-01-12	2026-05-05 09:25:00	10.10.5.102	LN350-E002	Ubuntu Server 24.04
17	srv-ci-01	1	Dell R750	32	128	1	2025-07-22	2026-05-07 13:50:00	10.10.6.111	DL750-CI01	Ubuntu Server 22.04
18	srv-ci-02	1	Dell R750	32	128	2	2025-07-22	2026-04-10 15:35:00	10.10.6.112	DL750-CI02	Ubuntu Server 22.04
19	srv-dr-01	8	HPE DL380	48	384	1	2024-12-03	2026-04-02 04:10:00	10.20.1.11	HP380-D001	Rocky Linux 9
20	srv-legacy-01	8	IBM x3650	16	64	5	2020-04-17	2025-12-15 10:00:00	10.20.1.12	IBM3650-L001	CentOS 7

Сервисы (примеры):

INSERT INTO services (id, name, server_id, port, status) VALUES
(1, 'billing-api', 1, 8080, 'RUNNING'),
(5, 'postgres-primary', 5, 5432, 'RUNNING'),
(7, 'redis-cache', 7, 6379, 'RUNNING'),
(13, 'ml-inference', 12, 8443, 'RUNNING'),
(21, 'legacy-crm', 20, 8088, 'STOPPED');

```
dc_equipment_rgr=# SELECT * FROM services;
```

id	name	server_id	port	status
1	billing-api	1	8080	RUNNING
2	billing-worker	2	8081	RUNNING
3	customer-portal	3	443	RUNNING
4	nginx-edge	4	443	DEGRADED
5	postgres-primary	5	5432	RUNNING
6	postgres-replica	6	5432	RUNNING
7	redis-cache	7	6379	RUNNING
8	redis-cache	8	6379	STOPPED
9	prometheus	9	9090	RUNNING
10	grafana	9	3000	RUNNING
11	loki	10	3100	RUNNING
12	backup-agent	11	9102	RUNNING
13	ml-inference	12	8443	RUNNING
14	ml-training	13	8444	DEGRADED
15	wireguard	14	51820	RUNNING
16	cdn-cache	15	8080	RUNNING
17	cdn-cache	16	8080	RUNNING
18	gitlab-runner	17	9252	RUNNING
19	jenkins-agent	18	50000	DEGRADED
20	dr-replication	19	9443	RUNNING
21	legacy-crm	20	8088	STOPPED
22	node-exporter	1	9100	RUNNING
23	node-exporter	5	9100	RUNNING
24	node-exporter	12	9100	RUNNING

(24 rows)

Инциденты (примеры):

INSERT INTO incidents (id, server_id, service_id, priority_id, title, detected_at, resolved_at) VALUES

(1, 8, 8, 3, 'Кэширующий слой Redis: превышение времени ответа', '2026-01-05 07:10:00', NULL),

(2, 4, 4, 2, 'API Gateway: рост количества HTTP 5xx ошибок', '2026-02-01 12:05:00', '2026-02-01 13:20:00'),

(3, 5, 5, 4, 'Кластер PostgreSQL: критическая задержка выполнения запросов', '2026-03-03 02:15:00', '2026-03-03 02:42:00'),

```
dc_equipment_rgr=# SELECT * FROM incidents;
```

id	server_id	service_id	priority_id	title	detected_at	resolved_at
1	8	8	3	Кэширующий слой Redis: превышение времени ответа	2026-01-05 07:10:00	
2	4	4	2	API Gateway: рост количества HTTP 5xx ошибок	2026-02-01 12:05:00	2026-02-01 13:20:00
3	5	5	4	Кластер PostgreSQL: критическая задержка выполнения запросов	2026-03-03 02:15:00	2026-03-03 02:42:00
4	6	6	3	Streaming-репликация: отставание от мастера	2026-03-03 02:20:00	2026-03-03 03:10:00
5	10	11	2	Logstash: падение пропускной способности обработки	2026-03-18 11:30:00	2026-03-18 12:05:00
6	12	13	4	Сервис инференса ML: таймауты GPU-воркеров	2026-04-06 09:00:00	2026-04-06 09:18:00
7	13	14	3	ML-пайплайн: задача обучения модели зависла в очереди	2026-04-08 15:45:00	
8	18	19	2	CI/CD агенты: увеличение времени сборки	2026-04-10 15:40:00	
9	20	21	1	CRM: падение после планового обновления ОС	2026-04-12 04:00:00	
10	1	1	2	Платёжный шлюз: всплеск ошибок валидации транзакций	2026-04-20 10:25:00	2026-04-20 10:55:00
11	2	2	1	Брокер сообщений: накопление необработанных задач в очереди	2026-04-22 13:10:00	2026-04-22 14:00:00
12	3	3	2	Портал: истечение срока действия TLS-сертификата	2026-04-25 08:35:00	2026-04-25 09:05:00
13	7	7	3	InnoDB: утечка памяти в кэше буфера	2026-04-29 21:00:00	2026-04-29 21:32:00
14	9	9	1	Prometheus: пропуски скрейпинга метрик	2026-05-02 06:10:00	2026-05-02 06:25:00
15	11	12	2	Система бэкапов: превышено число повторных попыток	2026-05-03 02:30:00	2026-05-03 03:25:00
16	14	15	3	VPN-шлюз: периодическая потеря пакетов	2026-05-06 18:15:00	
17	15	16	1	CDN: задержка прогрева кэша после деплоя	2026-05-08 09:45:00	2026-05-08 10:05:00
18	19	20	4	DR: разрыв канала асинхронной репликации	2026-05-09 23:40:00	2026-05-10 00:00:00

(18 rows)

Журнал обслуживания (примеры):

INSERT INTO maintenance_log (id, server_id, task_type, performed_at, description, downtime_minutes) VALUES

(1, 1, 'OS_PATCH', '2026-01-12 01:00:00', 'Установка патчей безопасности', 12),
(3, 3, 'FIRMWARE_UPDATE', '2026-01-19 02:00:00', 'Обновление прошивки
сетевого адаптера', 25),
(12, 12, 'GPU_DRIVER_UPDATE', '2026-04-06 09:30:00', 'Обновление драйвера
NVIDIA', 30);

```
dc_equipment_rgr=# SELECT * FROM maintenance_log;
```

id	server_id	task_type	performed_at	description	downtime_minutes
1	1	OS_PATCH	2026-01-12 01:00:00	Установка патчей безопасности	12
2	2	OS_PATCH	2026-01-12 01:30:00	Установка патчей безопасности	10
3	3	FIRMWARE_UPDATE	2026-01-19 02:00:00	Обновление прошивки сетевого адаптера	25
4	4	DIAGNOSTICS	2026-02-01 13:30:00	Диагностика сети после ошибок HTTP	35
5	5	STORAGE_CHECK	2026-03-03 03:00:00	Проверка состояния NVMe	18
6	6	REPLICATION_CHECK	2026-03-03 03:15:00	Исследование задержки репликации	20
7	7	RAM_UPGRADE	2026-03-20 01:00:00	Замена модуля памяти	45
8	8	HARDWARE_REPAIR	2026-01-05 08:00:00	Замена блока питания	90
9	9	OS_PATCH	2026-04-01 00:30:00	Установка патчей для стека мониторинга	8
10	10	DISK_EXPANSION	2026-04-21 06:00:00	Расширение раздела с логами	22
11	11	BACKUP_TEST	2026-04-30 03:00:00	Проверка восстановления из бэкапа	0
12	12	GPU_DRIVER_UPDATE	2026-04-06 09:30:00	Обновление драйвера NVIDIA	30
13	13	GPU_DIAGNOSTICS	2026-04-08 16:15:00	Диагностика очереди обучения	40
14	14	NETWORK_CHECK	2026-05-06 19:00:00	Проверка маршрутизации VPN	15
15	15	CACHE_RESTART	2026-05-08 10:00:00	Перезапуск кеш-сервиса	5
16	16	OS_PATCH	2026-05-08 10:30:00	Установка патчей на граничный узел	7
17	18	CI_AGENT_REPAIR	2026-04-10 16:00:00	Очистка и перезапуск раннера	28
18	19	DR_TEST	2026-05-10 00:15:00	Тренировка переключения репликации	32

(18 rows)

2.3 Визуализация структуры БД (diagram.png)

На основе созданной схемы построена ER-диаграмма, которая представлена на рисунке ниже.



Описание диаграммы:

Диаграмма отображает 8 таблиц со следующими связями:

Связь	Тип	Пояснение
racks → servers	один ко многим	Одна стойка может содержать много серверов
server_statuses → servers	один ко многим	Один статус может быть у многих серверов
servers → services	один ко многим	Один сервер может запускать много сервисов (с каскадным удалением)
servers → incidents	один ко многим	Один сервер может иметь много инцидентов
services → incidents	один ко многим	Один сервис может иметь много инцидентов (опционально)
incident_priorities → incidents	один ко многим	Один приоритет может быть у многих инцидентов
servers → maintenance_log	один ко многим	Один сервер может иметь много записей обслуживания
servers → servers_audit	один ко многим	Один сервер может иметь много записей аудита
server_statuses → servers_audit (old/new)	один ко многим	Справочник статусов используется для аудита

2.4 Хранимые функции (functions.sql)

Реализованы 4 хранимые функции на PL/pgSQL, решающие типовые задачи управления инфраструктурой.

Функция 1: Подсчёт активных инцидентов на сервере

```
CREATE OR REPLACE FUNCTION get_active_incidents_count(p_server_id INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM incidents
    WHERE server_id = p_server_id
    AND resolved_at IS NULL;

    RETURN v_count;
END;
$$;
```

Назначение: определяет, сколько нерешённых проблем в данный момент имеется на конкретном сервере.

Пример вызова:

```
SELECT get_active_incidents_count(8);
```

Результат: возвращает количество активных инцидентов для сервера srv-cache-02.

```
dc_equipment_rgr=# SELECT get_active_incidents_count(8);
get_active_incidents_count
-----
1
(1 row)

dc_equipment_rgr=# |
```

Функция 2: Расчёт энергопотребления стойки

```
CREATE OR REPLACE FUNCTION get_rack_power_usage(p_rack_id INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_power INT;
BEGIN
    SELECT COUNT(*) * 500
    INTO v_total_power
    FROM servers
    WHERE rack_id = p_rack_id;

    RETURN v_total_power;
END;
$$;
```

Назначение: оценивает суммарное энергопотребление всех серверов в стойке (в реальной системе здесь могла бы быть более сложная логика).

Пример вызова:

```
SELECT get_rack_power_usage(1);
```

Результат: возвращает примерное энергопотребление стойки RACK-A01 в ваттах.

```
dc_equipment_rgr=# SELECT get_rack_power_usage(1);
get_rack_power_usage
-----
3000
(1 row)
```

Функция 3: Суммарное время простоя за период

```
CREATE OR REPLACE FUNCTION get_server_downtime(
    p_server_id INT,
    p_start_date DATE,
    p_end_date DATE
)
RETURNS INTERVAL
LANGUAGE plpgsql
AS $$
```



```

DECLARE
    v_total_downtime INTERVAL;
BEGIN
    SELECT SUM(downtime_minutes * INTERVAL '1 minute')
    INTO v_total_downtime
    FROM maintenance_log
    WHERE server_id = p_server_id
    AND performed_at BETWEEN p_start_date AND p_end_date;

    RETURN COALESCE(v_total_downtime, INTERVAL '0');
END;
$$;

```

Назначение: суммирует время простоя сервера за указанный период на основе записей из журнала обслуживания.

Пример вызова:

```
SELECT get_server_downtime(8, '2026-01-01', '2026-12-31');
```

Результат: возвращает интервал времени (например, 01:30:00).

```

dc_equipment_rgr=# SELECT get_server_downtime(8, '2026-01-01', '2026-12-31');
get_server_downtime
-----
01:30:00
(1 row)

```

Функция 4: Проверка возможности добавления сервера в стойку

```

CREATE OR REPLACE FUNCTION can_add_server_to_rack(
    p_rack_id INT,
    p_cpu_cores INT,
    p_ram_gb INT
)
RETURNS BOOLEAN
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_cpu INT;
    v_total_ram INT;
    v_max_cpu INT := 1000;
    v_max_ram INT := 10000;
BEGIN
    SELECT COALESCE(SUM(cpu_cores), 0), COALESCE(SUM(ram_gb), 0)
    INTO v_total_cpu, v_total_ram
    FROM servers
    WHERE rack_id = p_rack_id;

    IF (v_total_cpu + p_cpu_cores) <= v_max_cpu
    AND (v_total_ram + p_ram_gb) <= v_max_ram THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
END;

```



```
END;  
$$;
```

Назначение: проверяет, не превышает ли добавление нового сервера установленные лимиты стойки по CPU и RAM.

Пример вызова:

```
SELECT can_add_server_to_rack(1, 16, 64);
```

Результат: возвращает TRUE или FALSE.

```
dc_equipment_rgr=# SELECT can_add_server_to_rack(1, 16, 64);  
can_add_server_to_rack  
-----  
t  
(1 row)
```

2.5 Триггеры (triggers.sql)

Реализованы 2 триггера для автоматизации бизнес-логики.

Триггер 1: Аудит изменения статуса сервера

Функция-обработчик:

```
CREATE OR REPLACE FUNCTION audit_server_status_change()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF OLD.status_id IS DISTINCT FROM NEW.status_id THEN  
        INSERT INTO servers_audit (  
            server_id,  
            old_status_id,  
            new_status_id,  
            changed_at,  
            changed_by  
        ) VALUES (  
            NEW.id,  
            OLD.status_id,  
            NEW.status_id,  
            CURRENT_TIMESTAMP,  
            CURRENT_USER  
        );  
    END IF;  
  
    RETURN NEW;  
END;  
$$;
```

Триггер:

```
CREATE TRIGGER trg_audit_server_status  
AFTER UPDATE OF status_id ON servers  
FOR EACH ROW  
EXECUTE FUNCTION audit_server_status_change();
```

Назначение: при каждом изменении поля status_id в таблице servers автоматически записывается запись в servers_audit с указанием старого статуса, нового статуса, времени изменения и имени пользователя, выполнившего изменение.

Триггер 2: Автообновление метки времени updated_at

Функция-обработчик:

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$;
```

Триггер:

```
CREATE TRIGGER trg_update_server_timestamp
BEFORE UPDATE ON servers
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

Назначение: перед обновлением любой строки в таблице servers автоматически устанавливает updated_at в текущее время, что позволяет отслеживать, когда была последняя модификация сервера.

2.6 Демонстрационный сценарий (demo.sql)

Сценарий demo.sql комплексно демонстрирует все возможности разработанной системы. Ниже приведены ключевые блоки с пояснениями.

Блок 1: Проверка наполненности базы

```
SELECT 'racks' AS table_name, COUNT(*) AS rows_count FROM racks
UNION ALL SELECT 'server_statuses', COUNT(*) FROM server_statuses
UNION ALL SELECT 'servers', COUNT(*) FROM servers
UNION ALL SELECT 'services', COUNT(*) FROM services
...
```

Назначение: показывает количество записей во всех таблицах, подтверждая корректность загрузки данных.

```
=====
1. Общая проверка наполнения базы
=====
```

table_name	rows_count
incident_priorities	4
incidents	18
maintenance_log	18
racks	8
server_statuses	5
servers	20
servers_audit	0
services	24
(8 rows)	

Блок 2: Вывод структуры, ограничений и индексов

```
SELECT table_name FROM information_schema.tables WHERE table_schema = 'public'
ORDER BY table_name;
```

```
SELECT tc.table_name, tc.constraint_name, tc.constraint_type FROM
information_schema.table_constraints tc ...;
```

```
SELECT tablename, indexname FROM pg_catalog.pg_indexes WHERE schemaname =
'public' ...;
```

Назначение: визуализирует метаданные базы — все таблицы, ограничения (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK) и созданные индексы.

```
=====
2. Структура: таблицы, ограничения и индексы
=====
```

table_name		

incident_priorities		
incidents		
maintenance_log		
racks		
server_statuses		
servers		
servers_audit		
services		
(8 rows)		

table_name	constraint_name	constraint_type

incident_priorities	2200_16430_1_not_null	CHECK
incident_priorities	2200_16430_2_not_null	CHECK
incident_priorities	incident_priorities_pkey	PRIMARY KEY
incident_priorities	incident_priorities_level_key	UNIQUE
incidents	2200_16439_1_not_null	CHECK
incidents	2200_16439_2_not_null	CHECK
incidents	2200_16439_4_not_null	CHECK
incidents	2200_16439_5_not_null	CHECK
incidents	incidents_check	CHECK
incidents	fk_incidents_priority	FOREIGN KEY
incidents	fk_incidents_server	FOREIGN KEY
incidents	fk_incidents_service	FOREIGN KEY
incidents	incidents_pkey	PRIMARY KEY
maintenance_log	2200_16448_1_not_null	CHECK
maintenance_log	2200_16448_2_not_null	CHECK
maintenance_log	2200_16448_3_not_null	CHECK
maintenance_log	maintenance_log_downtime_minutes_check	CHECK
maintenance_log	fk_maintenance_server	FOREIGN KEY
maintenance_log	maintenance_log_pkey	PRIMARY KEY
racks	2200_16386_1_not_null	CHECK
racks	2200_16386_2_not_null	CHECK
racks	2200_16386_3_not_null	CHECK
racks	racks_power_capacity_check	CHECK
racks	racks_pkey	PRIMARY KEY
racks	racks_name_key	UNIQUE

Блок 3: Связи серверов со стойками, статусами и сервисами

```
SELECT s.hostname, r.name AS rack, r.location, st.status_name,
       s.cpu_cores, s.ram_gb, COUNT(sv.id) AS services_count
FROM servers s
JOIN racks r ON s.rack_id = r.id
JOIN server_statuses st ON s.status_id = st.id
LEFT JOIN services sv ON s.id = sv.server_id
GROUP BY s.id, s.hostname, r.name, r.location, st.status_name, s.cpu_cores, s.ram_gb
ORDER BY r.name, s.hostname
LIMIT 20;
```

Назначение: показывает полную картину размещения серверов с подсчётом запущенных на них сервисов.

```
=====
3. Связи между таблицами: серверы, стойки, статусы, сервисы
=====
```

hostname	rack	location	status_name	cpu_cores	ram_gb	services_count
srv-app-01	RACK-A01	DC-1 / Hall A / Row 1	ACTIVE	32	128	2
srv-app-02	RACK-A01	DC-1 / Hall A / Row 1	ACTIVE	32	128	1
srv-ci-01	RACK-A01	DC-1 / Hall A / Row 1	ACTIVE	32	128	1
srv-ci-02	RACK-A01	DC-1 / Hall A / Row 1	MAINTENANCE	32	128	1
srv-web-01	RACK-A01	DC-1 / Hall A / Row 1	ACTIVE	24	64	1
srv-web-02	RACK-A01	DC-1 / Hall A / Row 1	MAINTENANCE	24	64	1
srv-cache-01	RACK-A02	DC-1 / Hall A / Row 1	ACTIVE	32	256	1
srv-cache-02	RACK-A02	DC-1 / Hall A / Row 1	OFFLINE	32	256	1
srv-db-01	RACK-A02	DC-1 / Hall A / Row 1	ACTIVE	64	512	2
srv-db-02	RACK-A02	DC-1 / Hall A / Row 1	ACTIVE	64	512	1
srv-bak-01	RACK-A03	DC-1 / Hall A / Row 2	ACTIVE	48	384	1
srv-log-01	RACK-A03	DC-1 / Hall A / Row 2	ACTIVE	32	256	1
srv-mon-01	RACK-A03	DC-1 / Hall A / Row 2	ACTIVE	16	64	2
srv-ai-01	RACK-B01	DC-1 / Hall B / Row 1	ACTIVE	128	1024	2
srv-ai-02	RACK-B01	DC-1 / Hall B / Row 1	PROVISIONING	128	1024	1
srv-edge-01	RACK-B02	DC-1 / Hall B / Row 1	ACTIVE	12	32	1
srv-edge-02	RACK-B02	DC-1 / Hall B / Row 1	ACTIVE	12	32	1
srv-vpn-01	RACK-B02	DC-1 / Hall B / Row 1	ACTIVE	16	64	1
srv-dr-01	RACK-D01	DR-Site / Hall D / Row 1	ACTIVE	48	384	1
srv-legacy-01	RACK-D01	DR-Site / Hall D / Row 1	DECOMMISSIONED	16	64	1

(20 rows)

Блок 4: Инциденты с приоритетами

```
SELECT i.title, s.hostname, sv.name AS service_name,
       ip.level, ip.response_minutes, i.detected_at, i.resolved_at
FROM incidents i
JOIN servers s ON i.server_id = s.id
LEFT JOIN services sv ON i.service_id = sv.id
JOIN incident_priorities ip ON i.priority_id = ip.id
ORDER BY i.detected_at DESC
LIMIT 10;
```

Назначение: выводит последние инциденты с информацией о сервере, сервисе и нормативах времени реакции.

```
=====
4. Инциденты с приоритетами и привязкой к серверу/сервису
=====
```

title	hostname	service_name	level	response_minutes	detected_at	resolved_at
Канал репликации DR прерван	srv-dr-01	dr-replication	CRITICAL	15	2026-05-09 23:40:00	2026-05-10 00:08:00
Задержка прогрева кеша CDN	srv-edge-01	cdn-cache	LOW	1440	2026-05-08 09:45:00	2026-05-08 10:05:00
Потеря пакетов VPN выше SLA	srv-vpn-01	wireguard	HIGH	60	2026-05-06 18:15:00	
Порог повторов бэкапа достигнут	srv-bak-01	backup-agent	MEDIUM	240	2026-05-03 02:30:00	2026-05-03 03:25:00
Пропуски скрейпинга Prometheus	srv-mon-01	prometheus	LOW	1440	2026-05-02 06:10:00	2026-05-02 06:25:00
Высокое потребление памяти кеша	srv-cache-01	redis-cache	HIGH	60	2026-04-29 21:00:00	2026-04-29 21:32:00
Предупреждение о TLS-сертификате портала	srv-web-01	customer-portal	MEDIUM	240	2026-04-25 08:35:00	2026-04-25 09:05:00
Очередь воркеров увеличилась	srv-app-02	billing-worker	LOW	1440	2026-04-22 13:10:00	2026-04-22 14:00:00
Уровень ошибок платежного API выше базового	srv-app-01	billing-api	MEDIUM	240	2026-04-20 10:25:00	2026-04-20 10:55:00
Legacy CRM остановилась после окна патчей	srv-legacy-01	legacy-crm	LOW	1440	2026-04-12 04:00:00	

(10 rows)

Блок 5: Аналитические запросы

Стойки с количеством серверов (агрегация):

```
SELECT r.name AS rack, COUNT(s.id) AS server_count, SUM(s.cpu_cores) AS total_cpu,
       SUM(s.ram_gb) AS total_ram_gb
FROM racks r
LEFT JOIN servers s ON r.id = s.rack_id
GROUP BY r.id, r.name
ORDER BY server_count DESC;
```

5. Аналитика: агрегации, HAVING, CTE, ORDER BY LIMIT

rack	server_count	total_cpu	total_ram_gb
RACK-A01	6	176	640
RACK-A02	4	192	1536
RACK-A03	3	96	704
RACK-B02	3	40	128
RACK-B01	2	256	2048
RACK-D01	2	64	448
RACK-C01	0		
RACK-C02	0		

(8 rows)

Стойки с более чем 5 серверами (HAVING):

```
SELECT r.name AS rack, COUNT(s.id) AS server_count
FROM racks r
LEFT JOIN servers s ON r.id = s.rack_id
GROUP BY r.id, r.name
HAVING COUNT(s.id) > 5
ORDER BY server_count DESC;
```

rack	server_count
RACK-A01	6

(1 row)

Топ-5 серверов по инцидентам (CTE + LIMIT):

```
WITH incident_stats AS (
  SELECT server_id, COUNT(*) AS incidents_count
  FROM incidents
  GROUP BY server_id
)
SELECT s.hostname, COALESCE(ist.incidents_count, 0) AS incidents_count
FROM servers s
LEFT JOIN incident_stats ist ON s.id = ist.server_id
ORDER BY incidents_count DESC, s.hostname
LIMIT 5;
```

hostname	incidents_count
srv-ai-01	1
srv-ai-02	1
srv-app-01	1
srv-app-02	1
srv-bak-01	1

(5 rows)

Блок 6: Вызов хранимых функций

```
SELECT get_active_incidents_count(8) AS active_incidents_on_srv_cache_02;  
SELECT get_rack_power_usage(1) AS rack_a01_estimated_power_usage_watts;  
SELECT get_server_downtime(8, '2026-01-01', '2026-12-31') AS srv_cache_02_downtime;  
SELECT can_add_server_to_rack(1, 16, 64) AS can_add_small_server_to_rack_a01;
```

```
=====
6. Демонстрация всех 4 функций
=====
```

```
active_incidents_on_srv_cache_02
```

```
-----
1  
(1 row)
```

```
rack_a01_estimated_power_usage_watts
```

```
-----
3000  
(1 row)
```

```
srv_cache_02_downtime
```

```
-----
01:30:00  
(1 row)
```

```
can_add_small_server_to_rack_a01
```

```
-----
t  
(1 row)
```

Блок 7: Демонстрация работы триггеров

```
SELECT s.hostname, st.status_name, s.updated_at  
FROM servers s  
JOIN server_statuses st ON s.status_id = st.id  
WHERE s.hostname = 'srv-app-01';
```

```
UPDATE servers SET status_id = (SELECT id FROM server_statuses WHERE  
status_name = 'MAINTENANCE')  
WHERE hostname = 'srv-app-01';
```

```
UPDATE servers SET status_id = (SELECT id FROM server_statuses WHERE  
status_name = 'ACTIVE')  
WHERE hostname = 'srv-app-01';
```

```
SELECT s.hostname, old_st.status_name AS old_status, new_st.status_name AS  
new_status,  
a.changed_at, a.changed_by  
FROM servers_audit a  
JOIN servers s ON a.server_id = s.id  
JOIN server_statuses old_st ON a.old_status_id = old_st.id  
JOIN server_statuses new_st ON a.new_status_id = new_st.id  
WHERE s.hostname = 'srv-app-01'
```

ORDER BY a.changed_at DESC;

=====

7. Демонстрация триггеров: аудит статуса и updated_at

=====

hostname	status_name	updated_at
srv-app-01	ACTIVE	2026-05-10 09:30:00

(1 row)

UPDATE 1
UPDATE 1

hostname	status_name	updated_at
srv-app-01	ACTIVE	2026-05-19 11:31:55.628313

(1 row)

hostname	old_status	new_status	changed_at	changed_by
srv-app-01	MAINTENANCE	ACTIVE	2026-05-19 11:31:55.628313	rgr_user
srv-app-01	ACTIVE	MAINTENANCE	2026-05-19 11:31:55.61563	rgr_user

(2 rows)

Блок 8: Проверка ON DELETE CASCADE

BEGIN;

INSERT INTO servers (hostname, rack_id, model, cpu_cores, ram_gb, status_id, purchase_date, ip_address)

VALUES ('srv-demo-cascade', 1, 'Demo Server', 4, 16, 1, CURRENT_DATE, '10.99.99.99');

INSERT INTO services (name, server_id, port, status)

SELECT 'demo-service', id, 8088, 'RUNNING' FROM servers WHERE hostname = 'srv-demo-cascade';

DELETE FROM servers WHERE hostname = 'srv-demo-cascade';

SELECT COUNT(*) FROM services WHERE name = 'demo-service';

ROLLBACK;

=====

8. Демонстрация ON DELETE CASCADE для services -> servers

=====

BEGIN

INSERT 0 1

INSERT 0 1

hostname	services_before_server_delete
srv-demo-cascade	1

(1 row)

DELETE 1

demo_services_after_server_delete

(1 row)

ROLLBACK

2.7 Аналитические запросы (queries.sql)

2.7.1 Простые SELECT-запросы (6 шт.)

1. Базовая информация о серверах

SELECT id, hostname, model, ram_gb, cpu_cores, purchase_date
FROM servers ORDER BY id;

dc_equipment_rgr=# SELECT id, hostname, model, ram_gb, cpu_cores, purchase_date dc_equipment_rgr=# FROM servers ORDER BY id;					
id	hostname	model	ram_gb	cpu_cores	purchase_date
1	srv-app-01	Dell R750	128	32	2024-03-14
2	srv-app-02	Dell R750	128	32	2024-03-14
3	srv-web-01	HPE DL360	64	24	2023-11-02
4	srv-web-02	HPE DL360	64	24	2023-11-02
5	srv-db-01	Dell R760	512	64	2025-01-19
6	srv-db-02	Dell R760	512	64	2025-01-19
7	srv-cache-01	Lenovo SR650	256	32	2024-08-08
8	srv-cache-02	Lenovo SR650	256	32	2024-08-08
9	srv-mon-01	Supermicro SYS-120	64	16	2023-05-30
10	srv-log-01	Supermicro SYS-220	256	32	2024-02-12
11	srv-bak-01	HPE DL380	384	48	2024-06-18
12	srv-ai-01	Dell XE9680	1024	128	2026-02-04
13	srv-ai-02	Dell XE9680	1024	128	2026-02-04
14	srv-vpn-01	HPE DL360	64	16	2022-09-10
15	srv-edge-01	Lenovo SE350	32	12	2026-01-12
16	srv-edge-02	Lenovo SE350	32	12	2026-01-12
17	srv-ci-01	Dell R750	128	32	2025-07-22
18	srv-ci-02	Dell R750	128	32	2025-07-22
19	srv-dr-01	HPE DL380	384	48	2024-12-03
20	srv-legacy-01	IBM x3650	64	16	2020-04-17

(20 rows)

2. Все инциденты с датами

SELECT id, title, detected_at, resolved_at
FROM incidents ORDER BY detected_at DESC;

dc_equipment_rgr=# SELECT id, title, detected_at, resolved_at dc_equipment_rgr=# FROM incidents ORDER BY detected_at DESC;			
id	title	detected_at	resolved_at
18	DR: разрыв канала асинхронной репликации	2026-05-09 23:40:00	2026-05-10 00:08:00
17	CDN: задержка прогрева кэша после деплоя	2026-05-08 09:45:00	2026-05-08 10:05:00
16	VPN-шлюз: периодическая потеря пакетов	2026-05-06 18:15:00	
15	Система бэкапов: превышено число повторных попыток	2026-05-03 02:30:00	2026-05-03 03:25:00
14	Prometheus: пропуск скрейпинга метрик	2026-05-02 06:10:00	2026-05-02 06:25:00
13	InnoDB: утечка памяти в кэше буфера	2026-04-29 21:00:00	2026-04-29 21:32:00
12	Портал: истечение срока действия TLS-сертификата	2026-04-25 08:35:00	2026-04-25 09:05:00
11	Брокер сообщений: накопление необработанных задач в очереди	2026-04-22 13:10:00	2026-04-22 14:00:00
10	Платёжный шлюз: всплеск ошибок валидации транзакций	2026-04-20 10:25:00	2026-04-20 10:55:00
9	CRM: падение после планового обновления ОС	2026-04-12 04:00:00	
8	CI/CD агенты: увеличение времени сборки	2026-04-10 15:40:00	
7	ML-пайплайн: задача обучения модели зависла в очереди	2026-04-08 15:45:00	
6	Сервис инференса ML: таймауты GPU-воркеров	2026-04-06 09:00:00	2026-04-06 09:18:00
5	Logstash: падение пропускной способности обработки	2026-03-18 11:30:00	2026-03-18 12:05:00
4	Streaming-репликация: отставание от мастера	2026-03-03 02:20:00	2026-03-03 03:10:00
3	Кластер PostgreSQL: критическая задержка выполнения запросов	2026-03-03 02:15:00	2026-03-03 02:42:00
2	API Gateway: рост количества HTTP 5xx ошибок	2026-02-01 12:05:00	2026-02-01 13:20:00
1	Кэширующий слой Redis: превышение времени ответа	2026-01-05 07:10:00	

(18 rows)

3. Журнал обслуживания

SELECT id, task_type, performed_at, downtime_minutes
FROM maintenance_log ORDER BY performed_at DESC;

```
dc_equipment_rgr=# SELECT id, task_type, performed_at, downtime_minutes
dc_equipment_rgr=# FROM maintenance_log ORDER BY performed_at DESC;
```

id	task_type	performed_at	downtime_minutes
18	DR_TEST	2026-05-10 00:15:00	32
16	OS_PATCH	2026-05-08 10:30:00	7
15	CACHE_RESTART	2026-05-08 10:00:00	5
14	NETWORK_CHECK	2026-05-06 19:00:00	15
11	BACKUP_TEST	2026-04-30 03:00:00	0
10	DISK_EXPANSION	2026-04-21 06:00:00	22
17	CI_AGENT_REPAIR	2026-04-10 16:00:00	28
13	GPU_DIAGNOSTICS	2026-04-08 16:15:00	40
12	GPU_DRIVER_UPDATE	2026-04-06 09:30:00	30
9	OS_PATCH	2026-04-01 00:30:00	8
7	RAM_UPGRADE	2026-03-20 01:00:00	45
6	REPLICATION_CHECK	2026-03-03 03:15:00	20
5	STORAGE_CHECK	2026-03-03 03:00:00	18
4	DIAGNOSTICS	2026-02-01 13:30:00	35
3	FIRMWARE_UPDATE	2026-01-19 02:00:00	25
2	OS_PATCH	2026-01-12 01:30:00	10
1	OS_PATCH	2026-01-12 01:00:00	12
8	HARDWARE_REPAIR	2026-01-05 08:00:00	90

(18 rows)

4. Стойки

SELECT id, name, location, power_capacity FROM racks ORDER BY name;

```
dc_equipment_rgr=# SELECT id, name, location, power_capacity FROM racks ORDER BY name;
```

id	name	location	power_capacity
1	RACK-A01	DC-1 / Hall A / Row 1	12000
2	RACK-A02	DC-1 / Hall A / Row 1	12000
3	RACK-A03	DC-1 / Hall A / Row 2	16000
4	RACK-B01	DC-1 / Hall B / Row 1	18000
5	RACK-B02	DC-1 / Hall B / Row 1	18000
6	RACK-C01	DC-2 / Hall C / Row 1	20000
7	RACK-C02	DC-2 / Hall C / Row 2	20000
8	RACK-D01	DR-Site / Hall D / Row 1	10000

(8 rows)

5. Сервисы

SELECT id, name, server_id, port, status FROM services ORDER BY name;

```
dc_equipment_rgr=# SELECT id, name, server_id, port, status FROM services ORDER BY name;
 id |      name      | server_id | port | status
-----+-----+-----+-----+-----
 12 | backup-agent   |      11 | 9102 | RUNNING
  1 | billing-api    |       1 | 8080 | RUNNING
  2 | billing-worker |       2 | 8081 | RUNNING
 17 | cdn-cache      |      16 | 8080 | RUNNING
 16 | cdn-cache      |      15 | 8080 | RUNNING
  3 | customer-portal |       3 | 443  | RUNNING
 20 | dr-replication |      19 | 9443 | RUNNING
 18 | gitlab-runner  |      17 | 9252 | RUNNING
 10 | grafana        |       9 | 3000 | RUNNING
 19 | jenkins-agent  |      18 | 50000 | DEGRADED
 21 | legacy-crm     |      20 | 8088 | STOPPED
 11 | loki           |       10 | 3100 | RUNNING
 13 | ml-inference   |      12 | 8443 | RUNNING
 14 | ml-training    |      13 | 8444 | DEGRADED
  4 | nginx-edge     |       4 | 443  | DEGRADED
 24 | node-exporter  |      12 | 9100 | RUNNING
 22 | node-exporter  |       1 | 9100 | RUNNING
 23 | node-exporter  |       5 | 9100 | RUNNING
  5 | postgres-primary |      5 | 5432 | RUNNING
  6 | postgres-replica |      6 | 5432 | RUNNING
  9 | prometheus     |       9 | 9090 | RUNNING
  8 | redis-cache    |       8 | 6379 | STOPPED
  7 | redis-cache    |       7 | 6379 | RUNNING
 15 | wireguard      |      14 | 51820 | RUNNING
(24 rows)
```

6. Приоритеты инцидентов

```
SELECT id, level, response_minutes
FROM incident_priorities ORDER BY response_minutes;
```

```
dc_equipment_rgr=# SELECT id, level, response_minutes FROM incident_priorities ORDER BY response_minutes;
 id | level | response_minutes
-----+-----+-----
  4 | CRITICAL |      15
  3 | HIGH |      60
  2 | MEDIUM |     240
  1 | LOW |    1440
(4 rows)
```

2.7.2 Запросы с фильтрацией WHERE (4 шт.)

7. Серверы с ОЗУ > 64 ГБ

```
SELECT hostname, model, ram_gb, cpu_cores
FROM servers WHERE ram_gb > 64 ORDER BY ram_gb DESC;
```

```
dc_equipment_rgr=# SELECT hostname, model, ram_gb, cpu_cores
dc_equipment_rgr=# FROM servers WHERE ram_gb > 64 ORDER BY ram_gb DESC;
 hostname |      model      | ram_gb | cpu_cores
-----+-----+-----+-----
 srv-ai-02 | Dell XE9680     |    1024 |      128
 srv-ai-01 | Dell XE9680     |    1024 |      128
 srv-db-02 | Dell R760       |     512 |       64
 srv-db-01 | Dell R760       |     512 |       64
 srv-bak-01 | HPE DL380       |     384 |       48
 srv-dr-01 | HPE DL380       |     384 |       48
 srv-log-01 | Supermicro SYS-220 |     256 |       32
 srv-cache-01 | Lenovo SR650    |     256 |       32
 srv-cache-02 | Lenovo SR650    |     256 |       32
 srv-app-01 | Dell R750       |     128 |       32
 srv-ci-01 | Dell R750       |     128 |       32
 srv-ci-02 | Dell R750       |     128 |       32
 srv-app-02 | Dell R750       |     128 |       32
(13 rows)
```

8. Серверы, купленные после 2025 года

```
SELECT hostname, model, purchase_date, os_name
FROM servers WHERE purchase_date > '2025-12-31' ORDER BY purchase_date DESC;
```

```
dc_equipment_rgr=# SELECT hostname, model, purchase_date, os_name
dc_equipment_rgr=# FROM servers WHERE purchase_date > '2025-12-31' ORDER BY purchase_date DESC;
 hostname | model | purchase_date | os_name
-----+-----+-----+-----
 srv-ai-01 | Dell XE9680 | 2026-02-04 | Ubuntu Server 24.04
 srv-ai-02 | Dell XE9680 | 2026-02-04 | Ubuntu Server 24.04
 srv-edge-01 | Lenovo SE350 | 2026-01-12 | Ubuntu Server 24.04
 srv-edge-02 | Lenovo SE350 | 2026-01-12 | Ubuntu Server 24.04
(4 rows)
```

9. Неразрешённые CRITICAL/HIGH инциденты

```
SELECT i.title, ip.level, i.detected_at
FROM incidents i JOIN incident_priorities ip ON i.priority_id = ip.id
WHERE i.resolved_at IS NULL AND ip.level IN ('CRITICAL', 'HIGH');
```

```
dc_equipment_rgr=# SELECT i.title, ip.level, i.detected_at
dc_equipment_rgr=# FROM incidents i JOIN incident_priorities ip ON i.priority_id = ip.id
dc_equipment_rgr=# WHERE i.resolved_at IS NULL AND ip.level IN ('CRITICAL', 'HIGH');
 title | level | detected_at
-----+-----+-----
 Кэширующий слой Redis: превышение времени ответа | HIGH | 2026-01-05 07:10:00
 ML-пайплайн: задача обучения модели зависла в очереди | HIGH | 2026-04-08 15:45:00
 VPN-шлюз: периодическая потеря пакетов | HIGH | 2026-05-06 18:15:00
(3 rows)

dc_equipment_rgr=# |
```

10. Серверы без обновлений более 90 дней

```
SELECT hostname, model, updated_at, status_id FROM servers
WHERE updated_at < CURRENT_DATE - INTERVAL '90 days'
ORDER BY updated_at ASC;
```

```
dc_equipment_rgr=# SELECT hostname, model, updated_at, status_id FROM servers
dc_equipment_rgr=# WHERE updated_at < CURRENT_DATE - INTERVAL '90 days'
dc_equipment_rgr=# ORDER BY updated_at ASC;
 hostname | model | updated_at | status_id
-----+-----+-----+-----
 srv-legacy-01 | IBM x3650 | 2025-12-15 10:00:00 | 5
 srv-cache-02 | Lenovo SR650 | 2026-01-05 07:00:00 | 3
 srv-vpn-01 | HPE DL360 | 2026-01-20 18:00:00 | 1
 srv-web-02 | HPE DL360 | 2026-02-01 12:00:00 | 2
(4 rows)
```

2.7.3 Запросы с JOIN (4 шт.)

11. Серверы + стойки + статусы

```
SELECT s.hostname, r.name AS rack, r.location, st.status_name
FROM servers s JOIN racks r ON s.rack_id = r.id
JOIN server_statuses st ON s.status_id = st.id;
```

```
dc_equipment_rgr=# SELECT s.hostname, r.name AS rack, r.location, st.status_name FROM servers s
ON s.status_id = st.id;
  hostname | rack | location | status_name
-----
srv-dr-01 | RACK-D01 | DR-Site / Hall D / Row 1 | ACTIVE
srv-vpn-01 | RACK-B02 | DC-1 / Hall B / Row 1 | ACTIVE
srv-edge-01 | RACK-B02 | DC-1 / Hall B / Row 1 | ACTIVE
srv-edge-02 | RACK-B02 | DC-1 / Hall B / Row 1 | ACTIVE
srv-ai-01 | RACK-B01 | DC-1 / Hall B / Row 1 | ACTIVE
srv-mon-01 | RACK-A03 | DC-1 / Hall A / Row 2 | ACTIVE
srv-log-01 | RACK-A03 | DC-1 / Hall A / Row 2 | ACTIVE
srv-bak-01 | RACK-A03 | DC-1 / Hall A / Row 2 | ACTIVE
srv-db-01 | RACK-A02 | DC-1 / Hall A / Row 1 | ACTIVE
srv-db-02 | RACK-A02 | DC-1 / Hall A / Row 1 | ACTIVE
srv-cache-01 | RACK-A02 | DC-1 / Hall A / Row 1 | ACTIVE
srv-app-01 | RACK-A01 | DC-1 / Hall A / Row 1 | ACTIVE
srv-app-02 | RACK-A01 | DC-1 / Hall A / Row 1 | ACTIVE
srv-web-01 | RACK-A01 | DC-1 / Hall A / Row 1 | ACTIVE
srv-ci-01 | RACK-A01 | DC-1 / Hall A / Row 1 | ACTIVE
srv-web-02 | RACK-A01 | DC-1 / Hall A / Row 1 | MAINTENANCE
srv-ci-02 | RACK-A01 | DC-1 / Hall A / Row 1 | MAINTENANCE
srv-cache-02 | RACK-A02 | DC-1 / Hall A / Row 1 | OFFLINE
srv-ai-02 | RACK-B01 | DC-1 / Hall B / Row 1 | PROVISIONING
srv-legacy-01 | RACK-D01 | DR-Site / Hall D / Row 1 | DECOMMISSIONED
(20 rows)
```

12. Неразрешённые инциденты + серверы + приоритеты

SELECT i.title, s.hostname, ip.level, ip.response_minutes, i.detected_at
FROM incidents i JOIN servers s ON i.server_id = s.id
JOIN incident_priorities ip ON i.priority_id = ip.id WHERE i.resolved_at IS NULL;

```
dc_equipment_rgr=# SELECT i.title, s.hostname, ip.level, ip.response_minutes, i.detected_at
dc_equipment_rgr=# FROM incidents i JOIN servers s ON i.server_id = s.id
dc_equipment_rgr=# JOIN incident_priorities ip ON i.priority_id = ip.id WHERE i.resolved_at IS NULL;
  title | hostname | level | response_minutes | detected_at
-----
Кэширующий слой Redis: превышение времени ответа | srv-cache-02 | HIGH | 60 | 2026-01-05 07:10:00
ML-пайплайн: задача обучения модели зависла в очереди | srv-ai-02 | HIGH | 60 | 2026-04-08 15:45:00
CI/CD агенты: увеличение времени сборки | srv-ci-02 | MEDIUM | 240 | 2026-04-10 15:40:00
CRM: падение после планового обновления ОС | srv-legacy-01 | LOW | 1440 | 2026-04-12 04:00:00
VPN-шлюз: периодическая потеря пакетов | srv-vpn-01 | HIGH | 60 | 2026-05-06 18:15:00
(5 rows)
```

13. Сервисы + серверы + стойки

SELECT sv.name AS service_name, s.hostname, r.name AS rack, sv.port, sv.status
FROM services sv JOIN servers s ON sv.server_id = s.id JOIN racks r ON s.rack_id = r.id;

```
dc_equipment_rgr=# SELECT sv.name AS service_name, s.hostname, r.name AS rack, sv.port, sv.status
dc_equipment_rgr=# FROM services sv JOIN servers s ON sv.server_id = s.id JOIN racks r ON s.rack_id = r.id;
  service_name | hostname | rack | port | status
-----
billing-api | srv-app-01 | RACK-A01 | 8080 | RUNNING
billing-worker | srv-app-02 | RACK-A01 | 8081 | RUNNING
customer-portal | srv-web-01 | RACK-A01 | 443 | RUNNING
nginx-edge | srv-web-02 | RACK-A01 | 443 | DEGRADED
postgres-primary | srv-db-01 | RACK-A02 | 5432 | RUNNING
postgres-replica | srv-db-02 | RACK-A02 | 5432 | RUNNING
redis-cache | srv-cache-01 | RACK-A02 | 6379 | RUNNING
redis-cache | srv-cache-02 | RACK-A02 | 6379 | STOPPED
prometheus | srv-mon-01 | RACK-A03 | 9090 | RUNNING
grafana | srv-mon-01 | RACK-A03 | 3000 | RUNNING
loki | srv-log-01 | RACK-A03 | 3100 | RUNNING
backup-agent | srv-bak-01 | RACK-A03 | 9102 | RUNNING
ml-inference | srv-ai-01 | RACK-B01 | 8443 | RUNNING
ml-training | srv-ai-02 | RACK-B01 | 8444 | DEGRADED
wireguard | srv-vpn-01 | RACK-B02 | 51820 | RUNNING
cdn-cache | srv-edge-01 | RACK-B02 | 8080 | RUNNING
cdn-cache | srv-edge-02 | RACK-B02 | 8080 | RUNNING
gitlab-runner | srv-ci-01 | RACK-A01 | 9252 | RUNNING
jenkins-agent | srv-ci-02 | RACK-A01 | 50000 | DEGRADED
dr-replication | srv-dr-01 | RACK-D01 | 9443 | RUNNING
legacy-crm | srv-legacy-01 | RACK-D01 | 8088 | STOPPED
node-exporter | srv-app-01 | RACK-A01 | 9100 | RUNNING
node-exporter | srv-db-01 | RACK-A02 | 9100 | RUNNING
node-exporter | srv-ai-01 | RACK-B01 | 9100 | RUNNING
(24 rows)
```


14. Аудит + серверы + статусы

```
SELECT s.hostname, old_st.status_name AS old_status, new_st.status_name AS  
new_status, a.changed_at, a.changed_by  
FROM servers_audit a JOIN servers s ON a.server_id = s.id  
JOIN server_statuses old_st ON a.old_status_id = old_st.id  
JOIN server_statuses new_st ON a.new_status_id = new_st.id  
ORDER BY a.changed_at DESC LIMIT 50;
```

```
dc_equipment_rgr=# SELECT s.hostname, old_st.status_name AS old_status, new_st.status_name AS new_status, a.changed_at, a.changed_b  
y  
dc_equipment_rgr=# FROM servers_audit a JOIN servers s ON a.server_id = s.id  
dc_equipment_rgr=# JOIN server_statuses old_st ON a.old_status_id = old_st.id  
dc_equipment_rgr=# JOIN server_statuses new_st ON a.new_status_id = new_st.id  
dc_equipment_rgr=# ORDER BY a.changed_at DESC LIMIT 50;  
hostname | old_status | new_status | changed_at | changed_by  
-----  
srv-app-01 | MAINTENANCE | ACTIVE | 2026-05-19 13:45:51.879075 | rgr_user  
srv-app-01 | ACTIVE | MAINTENANCE | 2026-05-19 13:45:51.874474 | rgr_user  
(2 rows)
```

2.7.4 Запросы с агрегацией GROUP BY (6 шт.)

15. Количество серверов по стойкам

```
SELECT r.name AS rack, COUNT(s.id) AS server_count  
FROM racks r LEFT JOIN servers s ON r.id = s.rack_id  
GROUP BY r.name ORDER BY server_count DESC;
```

```
dc_equipment_rgr=# SELECT r.name AS rack, COUNT(s.id) AS server_count  
dc_equipment_rgr=# FROM racks r LEFT JOIN servers s ON r.id = s.rack_id  
dc_equipment_rgr=# GROUP BY r.name ORDER BY server_count DESC;  
rack | server_count  
-----  
RACK-A01 | 6  
RACK-A02 | 4  
RACK-A03 | 3  
RACK-B02 | 3  
RACK-D01 | 2  
RACK-B01 | 2  
RACK-C02 | 0  
RACK-C01 | 0  
(8 rows)
```

16. Среднее время простоя по типу работ

```
SELECT task_type, AVG(downtime_minutes) AS avg_downtime_minutes, COUNT(*) AS  
operations_count  
FROM maintenance_log GROUP BY task_type  
ORDER BY avg_downtime_minutes DESC;
```

```
dc_equipment_rgr=# SELECT task_type, AVG(downtime_minutes) AS avg_downtime_minutes, COUNT(*) AS operations_count  
dc_equipment_rgr=# FROM maintenance_log GROUP BY task_type  
dc_equipment_rgr=# ORDER BY avg_downtime_minutes DESC;  
task_type | avg_downtime_minutes | operations_count  
-----  
HARDWARE_REPAIR | 90.0000000000000000 | 1  
RAM_UPGRADE | 45.0000000000000000 | 1  
GPU_DIAGNOSTICS | 40.0000000000000000 | 1  
DIAGNOSTICS | 35.0000000000000000 | 1  
DR_TEST | 32.0000000000000000 | 1  
GPU_DRIVER_UPDATE | 30.0000000000000000 | 1  
CI_AGENT_REPAIR | 28.0000000000000000 | 1  
FIRMWARE_UPDATE | 25.0000000000000000 | 1  
DISK_EXPANSION | 22.0000000000000000 | 1  
REPLICATION_CHECK | 20.0000000000000000 | 1  
STORAGE_CHECK | 18.0000000000000000 | 1  
NETWORK_CHECK | 15.0000000000000000 | 1  
OS_PATCH | 9.2500000000000000 | 4  
CACHE_RESTART | 5.0000000000000000 | 1  
BACKUP_TEST | 0.0000000000000000 | 1  
(15 rows)
```

17. Количество инцидентов по приоритетам

```
SELECT ip.level, COUNT(i.id) AS incident_count
FROM incident_priorities ip LEFT JOIN incidents i ON ip.id = i.priority_id
GROUP BY ip.level ORDER BY incident_count DESC;
```

```
dc_equipment_rgr=# SELECT ip.level, COUNT(i.id) AS incident_count
dc_equipment_rgr=# FROM incident_priorities ip LEFT JOIN incidents i ON ip.id = i.priority_id
dc_equipment_rgr=# GROUP BY ip.level ORDER BY incident_count DESC;
 level | incident_count
-----|-----
 MEDIUM | 6
  HIGH  | 5
  LOW   | 4
 CRITICAL | 3
(4 rows)
```

18. Суммарное количество ядер по моделям

```
SELECT model, SUM(cpu_cores) AS total_cores, COUNT(*) AS servers_count
FROM servers GROUP BY model ORDER BY total_cores DESC;
```

```
dc_equipment_rgr=# SELECT model, SUM(cpu_cores) AS total_cores, COUNT(*) AS servers_count FROM servers GROUP BY model ORDER BY total_cores DESC;
 model | total_cores | servers_count
-----|-----|-----
 Dell XE9680 | 256 | 2
 Dell R750 | 128 | 4
 Dell R760 | 128 | 2
 HPE DL380 | 96 | 2
 HPE DL360 | 64 | 3
 Lenovo SR650 | 64 | 2
 Supermicro SYS-220 | 32 | 1
 Lenovo SE350 | 24 | 2
 Supermicro SYS-120 | 16 | 1
 IBM x3650 | 16 | 1
(10 rows)
```

19. Общий объём ОЗУ по статусам

```
SELECT st.status_name, SUM(s.ram_gb) AS total_ram_gb, AVG(s.ram_gb) AS avg_ram_gb
FROM servers s JOIN server_statuses st ON s.status_id = st.id
GROUP BY st.status_name ORDER BY total_ram_gb DESC;
```

```
dc_equipment_rgr=# SELECT st.status_name, SUM(s.ram_gb) AS total_ram_gb, AVG(s.ram_gb) AS avg_ram_gb
dc_equipment_rgr=# FROM servers s JOIN server_statuses st ON s.status_id = st.id
dc_equipment_rgr=# GROUP BY st.status_name ORDER BY total_ram_gb DESC;
 status_name | total_ram_gb | avg_ram_gb
-----|-----|-----
 ACTIVE | 3968 | 264.53333333333333
 PROVISIONING | 1024 | 1024.00000000000000
 OFFLINE | 256 | 256.00000000000000
 MAINTENANCE | 192 | 96.00000000000000
 DECOMMISSIONED | 64 | 64.00000000000000
(5 rows)
```

20. Количество обслуживаний по серверам

```
SELECT s.hostname, COUNT(m.id) AS maintenance_count, SUM(m.downtime_minutes) AS total_downtime
FROM servers s LEFT JOIN maintenance_log m ON s.id = m.server_id
GROUP BY s.id, s.hostname ORDER BY maintenance_count DESC;
```

```
dc_equipment_rgr=# SELECT s.hostname, COUNT(m.id) AS maintenance_count, SUM(m.downtime_minutes) AS total_downtime
dc_equipment_rgr=# FROM servers s LEFT JOIN maintenance_log m ON s.id = m.server_id
dc_equipment_rgr=# GROUP BY s.id, s.hostname ORDER BY maintenance_count DESC;
```

hostname	maintenance_count	total_downtime
srv-cache-02	1	90
srv-log-01	1	22
srv-db-02	1	20
srv-vpn-01	1	15
srv-ai-02	1	40
srv-app-02	1	10
srv-edge-02	1	7
srv-bak-01	1	0
srv-mon-01	1	8
srv-cache-01	1	45
srv-edge-01	1	5
srv-ai-01	1	30
srv-dr-01	1	32
srv-web-01	1	25
srv-ci-02	1	28
srv-web-02	1	35
srv-app-01	1	12
srv-db-01	1	18
srv-ci-01	0	
srv-legacy-01	0	

(20 rows)

2.7.5 Запросы с HAVING (2 шт.)

21. Стойки с более чем 5 серверами

```
SELECT r.name AS rack, COUNT(s.id) AS server_count
```

```
FROM racks r LEFT JOIN servers s ON r.id = s.rack_id
```

```
GROUP BY r.id, r.name HAVING COUNT(s.id) > 5 ORDER BY server_count DESC;
```

```
dc_equipment_rgr=# SELECT r.name AS rack, COUNT(s.id) AS server_count
dc_equipment_rgr=# FROM racks r LEFT JOIN servers s ON r.id = s.rack_id
dc_equipment_rgr=# GROUP BY r.id, r.name HAVING COUNT(s.id) > 5 ORDER BY server_count DESC;
```

rack	server_count
RACK-A01	6

(1 row)

22. Модели серверов со средним ОЗУ больше 32 ГБ

```
SELECT model, AVG(ram_gb) AS avg_ram, COUNT(*) AS count
```

```
FROM servers GROUP BY model HAVING AVG(ram_gb) > 32
```

```
ORDER BY avg_ram DESC;
```

```
dc_equipment_rgr=# SELECT model, AVG(ram_gb) AS avg_ram, COUNT(*) AS count FROM servers GROUP BY model HAVING AVG(ram_gb) > 32 ORDE
R BY avg_ram DESC;
```

model	avg_ram	count
Dell XE9680	1024.0000000000000000	2
Dell R760	512.0000000000000000	2
HPE DL380	384.0000000000000000	2
Supermicro SYS-220	256.0000000000000000	1
Lenovo SR650	256.0000000000000000	2
Dell R750	128.0000000000000000	4
HPE DL360	64.0000000000000000	3
Supermicro SYS-120	64.0000000000000000	1
IBM x3650	64.0000000000000000	1

(9 rows)

2.7.6 Запросы с CTE (2 шт.)

23. Топ-3 сервера по количеству инцидентов

```
WITH incident_stats AS (
```

```
SELECT server_id, COUNT(*) AS total_incidents
```

```
FROM incidents GROUP BY server_id)
```

```
SELECT s.hostname, s.model, COALESCE(incident_stats.total_incidents, 0) AS
incident_count
```

```
FROM servers s LEFT JOIN incident_stats ON s.id = incident_stats.server_id
```

```
ORDER BY incident_count DESC LIMIT 3;
```



```
dc_equipment_rgr=# WITH incident_stats AS (
dc_equipment_rgr=# SELECT server_id, COUNT(*) AS total_incidents
dc_equipment_rgr=# FROM incidents GROUP BY server_id)
dc_equipment_rgr=# SELECT s.hostname, s.model, COALESCE(incident_stats.total_incidents, 0) AS incident_count
dc_equipment_rgr=# FROM servers s LEFT JOIN incident_stats ON s.id = incident_stats.server_id
dc_equipment_rgr=# ORDER BY incident_count DESC LIMIT 3;
 hostname | model | incident_count
-----|-----|-----
 srv-mon-01 | Supermicro SYS-120 | 1
 srv-edge-01 | Lenovo SE350 | 1
 srv-bak-01 | HPE DL380 | 1
(3 rows)
```

24. Серверы с простоем выше среднего

```
WITH avg_downtime AS (
SELECT AVG(downtime_minutes) AS avg_all FROM maintenance_log)
SELECT s.hostname, SUM(m.downtime_minutes) AS total_downtime
FROM servers s JOIN maintenance_log m ON s.id = m.server_id
CROSS JOIN avg_downtime GROUP BY s.id, s.hostname, avg_downtime.avg_all
HAVING SUM(m.downtime_minutes) > avg_downtime.avg_all
ORDER BY total_downtime DESC;
```

```
dc_equipment_rgr=# WITH avg_downtime AS (
dc_equipment_rgr=# SELECT AVG(downtime_minutes) AS avg_all FROM maintenance_log)
dc_equipment_rgr=# SELECT s.hostname, SUM(m.downtime_minutes) AS total_downtime
dc_equipment_rgr=# FROM servers s JOIN maintenance_log m ON s.id = m.server_id
dc_equipment_rgr=# CROSS JOIN avg_downtime GROUP BY s.id, s.hostname, avg_downtime.avg_all HAVING SUM(m.downtime_minutes) > avg_downtime.avg_all
dc_equipment_rgr=# ORDER BY total_downtime DESC;
 hostname | total_downtime
-----|-----
 srv-cache-02 | 90
 srv-cache-01 | 45
 srv-ai-02 | 40
 srv-web-02 | 35
 srv-dr-01 | 32
 srv-ai-01 | 30
 srv-ci-02 | 28
 srv-web-01 | 25
(8 rows)
```

2.7.7 Запросы с ORDER BY + LIMIT (2 шт.)

25. Последние 5 инцидентов

```
SELECT i.title, s.hostname, ip.level, i.detected_at
FROM incidents i JOIN servers s ON i.server_id = s.id
JOIN incident_priorities ip ON i.priority_id = ip.id
ORDER BY i.detected_at DESC LIMIT 5;
```

```
dc_equipment_rgr=# SELECT i.title, s.hostname, ip.level, i.detected_at
dc_equipment_rgr=# FROM incidents i JOIN servers s ON i.server_id = s.id
dc_equipment_rgr=# JOIN incident_priorities ip ON i.priority_id = ip.id
dc_equipment_rgr=# ORDER BY i.detected_at DESC LIMIT 5;
 title | hostname | level | detected_at
-----|-----|-----|-----
 DR: разрыв канала асинхронной репликации | srv-dr-01 | CRITICAL | 2026-05-09 23:40:00
 CDN: задержка прогрева кэша после деплоя | srv-edge-01 | LOW | 2026-05-08 09:45:00
 VPN-шлюз: периодическая потеря пакетов | srv-vpn-01 | HIGH | 2026-05-06 18:15:00
 Система бэкапов: превышено число повторных попыток | srv-bak-01 | MEDIUM | 2026-05-03 02:30:00
 Prometheus: пропуски скрейпинга метрик | srv-mon-01 | LOW | 2026-05-02 06:10:00
(5 rows)
```

26. Топ-5 серверов с наибольшим объёмом ОЗУ

```
SELECT hostname, model, ram_gb, os_name
FROM servers ORDER BY ram_gb DESC LIMIT 5;
```

```
dc_equipment_rgr=# SELECT hostname, model, ram_gb, os_name
dc_equipment_rgr=# FROM servers ORDER BY ram_gb DESC LIMIT 5;
 hostname |      model      | ram_gb |      os_name
-----|-----|-----|-----
 srv-ai-02 | Dell XE9680     | 1024   | Ubuntu Server 24.04
 srv-ai-01 | Dell XE9680     | 1024   | Ubuntu Server 24.04
 srv-db-01 | Dell R760       | 512    | Rocky Linux 9
 srv-db-02 | Dell R760       | 512    | Rocky Linux 9
 srv-bak-01 | HPE DL380       | 384    | Ubuntu Server 24.04
(5 rows)
```

2.8 Запуск через Docker Compose

Для обеспечения воспроизводимости окружения и упрощения проверки создан файл docker-compose.yaml.

2.8.1 Файл конфигурации

services:

postgres:

image: postgres:16-alpine

container_name: dc_equipment_postgres

environment:

POSTGRES_DB: dc_equipment_rgr

POSTGRES_USER: rgr_user

POSTGRES_PASSWORD: rgr_password

ports:

- "5433:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./schema.sql:/docker-entrypoint-initdb.d/01_schema.sql:ro

- ./data.sql:/docker-entrypoint-initdb.d/02_data.sql:ro

- ./functions.sql:/docker-entrypoint-initdb.d/03_functions.sql:ro

- ./triggers.sql:/docker-entrypoint-initdb.d/04_triggers.sql:ro

- ./queries.sql:/workspace/queries.sql:ro

- ./demo.sql:/workspace/demo.sql:ro

healthcheck:

test: ["CMD-SHELL", "pg_isready -U rgr_user -d dc_equipment_rgr"]

interval: 5s

timeout: 5s

retries: 10

volumes:

postgres_data:

```
PS C:\Users\sergx\OneDrive\Desktop\db_rgr> docker-compose up -d
time="2026-05-19T20:13:28+07:00" level=warning msg="No services to build"
[+] up 3/3
✓Network db_rgr_default          Created      0.0s
✓Volume db_rgr_postgres_data    Created      0.0s
✓Container dc_equipment_postgres Created      0.1s
PS C:\Users\sergx\OneDrive\Desktop\db_rgr> |
```

2.8.2 Порядок запуска

Шаг	Команда	Назначение
1	<code>docker compose up -d</code>	Запуск контейнера в фоновом режиме
2	<code>docker compose ps</code>	Проверка статуса контейнера
3	<code>docker compose exec postgres psql -U rgr_user -d dc_equipment_rgr</code>	Подключение к интерактивной консоли PostgreSQL
4	<code>docker compose exec postgres psql -U rgr_user -d dc_equipment_rgr -f /workspace/queries.sql</code>	Выполнение аналитических запросов
5	<code>docker compose exec postgres psql -U rgr_user -d dc_equipment_rgr -f /workspace/demo.sql</code>	Запуск комплексной демонстрации
6	<code>docker compose down</code>	Остановка контейнера (данные сохраняются)
7	<code>docker compose down -v</code>	Остановка с удалением тома (полная очистка)

2.8.3 Автоматическая инициализация

При первом запуске контейнера PostgreSQL автоматически выполняет SQL-файлы из каталога `/docker-entrypoint-initdb.d/` в алфавитном порядке:

1. `01_schema.sql` — создание структуры таблиц
2. `02_data.sql` — наполнение тестовыми данными
3. `03_functions.sql` — создание хранимых функций
4. `04_triggers.sql` — создание триггеров

Благодаря такому порядку обеспечивается корректная последовательность создания объектов базы данных.

2.9 Результаты выполнения

В результате выполнения работы создана полностью функциональная база данных для управления ИТ-инфраструктурой дата-центра, включающая:

- **8 таблиц**, связанных внешними ключами;
- **97+ тестовых записей**, демонстрирующих все сценарии использования;
- **13 индексов** для оптимизации производительности запросов;
- **4 хранимые функции** на PL/pgSQL;
- **2 триггера** (аудит статусов и автообновление меток времени);

- **26 аналитических SQL-запросов** с использованием WHERE, JOIN, GROUP BY, HAVING, CTE, ORDER BY LIMIT;
- **Docker Compose** для воспроизводимого развёртывания.

Все скрипты протестированы и работают корректно.

Демонстрационный сценарий demo.sql подтверждает работоспособность всех механизмов.

3. Выводы

В данной расчетно-графической работе была спроектирована и реализована реляционная база данных для управления IT-инфраструктурой дата-центра. Разработанная схема включает в себя восемь таблиц, которые описывают основные сущности предметной области: стойки, серверы, сервисы, инциденты, статусы, приоритеты, журнал обслуживания и аудит изменений. Все таблицы связаны внешними ключами, для большинства полей заданы ограничения, а для сервисов настроено каскадное удаление.

База данных была развернута в СУБД PostgreSQL. Для её создания и наполнения были подготовлены SQL-скрипты, которые последовательно создают структуру, добавляют тестовые данные (более 90 записей), индексы для ускорения работы, а также хранимые функции и триггеры. Одна из реализованных функций позволяет подсчитать активные инциденты на сервере, а триггер автоматически ведет журнал изменений статуса оборудования.

Для проверки работоспособности был составлен набор из 26 аналитических запросов, демонстрирующих выборки с фильтрацией, соединениями, группировкой и использованием обобщенных табличных выражений. Кроме того, был подготовлен демонстрационный сценарий, который наглядно показывает работу всех механизмов, включая триггеры и каскадное удаление. Для упрощения запуска и обеспечения воспроизводимости окружения все компоненты были упакованы в Docker Compose.

В итоге была создана полностью работоспособная база данных, готовая к использованию в задачах учёта и мониторинга серверного оборудования. Все поставленные в работе цели выполнены в полном объёме.